

705.641.81: Natural Language Processing Self-Supervised Models

Homework 3: Building Your Neural Network!

For homework deadline and collaboration policy, please see our Canvas page.

Name: Angela Zhao

Collaborators, if any: Yuzi Li

Sources used for your homework, if any: _____

This assignment is focusing on understanding the fundamental properties of neural networks and their training.

Homework goals: After completing this homework, you should be comfortable with:

- thinking about neural networks
- key implementation details of NNs, particularly in PyTorch,
- explaining and deriving Backpropagation,
- debugging your neural network in case it faces any failures.

1. Concepts, intuitions and big picture

1. Option 1
2. False
3. True
4. Option 4

5. Dropout randomly disables a set of neurons in a layer. In training, we want to do this primarily to prevent overfitting. It prevents the model from being over-reliant on specific nodes, and improves accuracy on new data. In evaluation, we want to use the full capacity of the model with all the neurons. Furthermore, since neurons disable randomly, the outputs of the evaluation are not consistent.

6. If we initialize parameters of a neural net with a constant, we end up with a symmetry problem. All neurons in that layer will produce the same output and also have the same gradient during back propagation. As a result, all neurons will learn the same features and be identical throughout training. The result is that the network will act as if each layer only has one neuron, and the model may not be able to learn more complex, diverse patterns in the data.

7. Not a good idea. Large positive weights will push the sigmoid function to its maximum value of 1, i.e. neuron saturation. This will lead to a vanishing gradient as the derivative approaches 0, so weights are not meaningfully updated during training.

8. Residual connections make optimization easier for deeper nets and mitigate the vanishing/exploding gradient problem by providing a shortcut for gradients to flow backwards and bypass some layers of the neural net during training. This allows us to train deeper neural nets.

9. Option 1
10. Option 1

2. Revisiting Jacobians

	<u>Shape</u>	<u>J(w)</u>
1.	$1 \times d$	$[0, \dots, 0]$
2.	$1 \times d$	$[2w_1, \dots, 2w_d]$
3.	$1 \times d$	$[x_{i1}, \dots, x_{id}] = \tilde{x}_i$
4.	$n \times d$	X
5.	$d \times d$	$I_{d \times d}$
6.	$d \times d$	$\text{diag}(2w)$
7.	$(n \times k) \times (k \times d)$	$\begin{aligned} J(w)_{ijk\ell} &= \frac{\partial f(w)_{ij}}{\partial w_{k\ell}} = \frac{\partial (x_i^T w_j)}{\partial w_{k\ell}} \\ &= \begin{cases} x_{i\ell} & \text{if } j=k \\ 0 & \text{otw} \end{cases} \end{aligned}$

3. Concepts, intuitions and big picture

1. Activation functions are used to introduce non-linearity into neural networks so that it can learn more complex, non-linear relationships in the data.

2. Sigmoid : $\sigma(x) = \frac{1}{1+e^{-x}}$

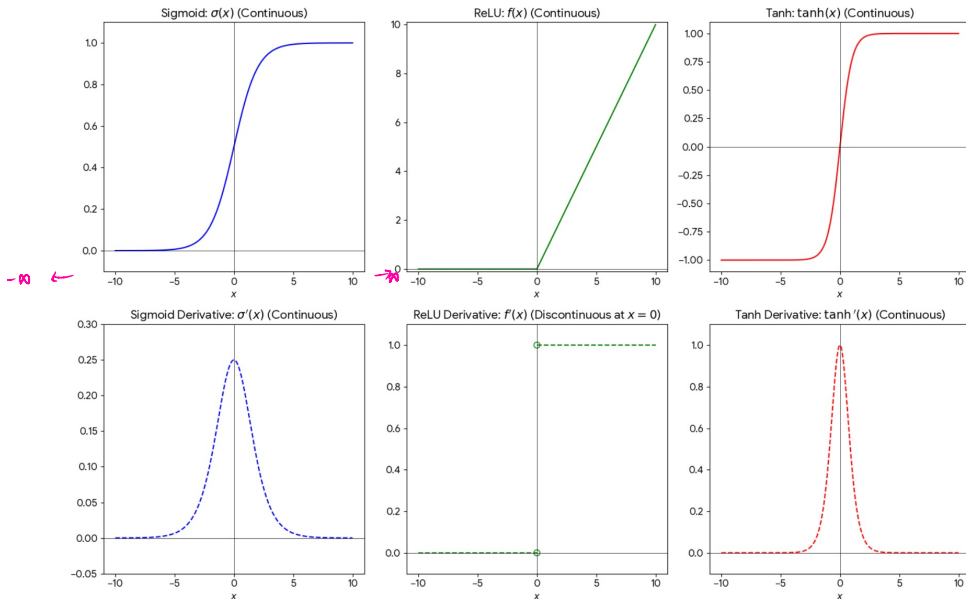
$$\frac{d}{dx} \sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \sigma(x)(1-\sigma(x))$$

$$\text{ReLU} : \text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undef.} & \text{if } x = 0 \end{cases}$$

$$\text{Tanh} : \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) = \left(\frac{2}{e^x + e^{-x}} \right)^2$$



Domain is $(-\infty, \infty)$. Asymptotic behaviour. X-axis truncated for visibility.

3. The vanishing gradient problem occurs when gradients used to update the weights of a neural become exponentially small. As the gradients are propagated from the output layer to the early layers, many matrix multiplications are used in the calculation (chain rule). As a result, these gradients do not meaningfully update the weights and the neural net does not learn. The sigmoid function is especially sensitive to this issue, as its derivative falls on the range of $[0, 0.25]$ and multiplying small decimals causes the gradient to decay quickly. The tanh activation function is also susceptible to this, although not as severely, as derivatives fall on $[0, 1]$.

4. Non zero-centered activation functions like sigmoid which is always positive constrains the directions the gradient can move during back propagation. Zero-centered activation functions like tanh don't force all weight updates in a layer to be either entirely positive or negative, so the algorithm converges faster.

5. Softmax: $\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$

$$\begin{aligned} \frac{d}{dz_j} \sigma(\vec{z})_i &= -e^{z_i} \left(\sum_{j=1}^K e^{z_j} \right)^{-2} e^{z_j} + \delta_{ij} \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \\ &= -\sigma(\vec{z})_i \frac{e^{z_j}}{\sum_{j=1}^K e^{z_j}} + \delta_{ij} \sigma(\vec{z})_i \\ &= \sigma(\vec{z})_i (\delta_{ij} - \sigma(\vec{z})_j) \end{aligned}$$

6. $[J_\sigma(\vec{z})]_{ij} = -\sigma(\vec{z})_i \sigma(\vec{z})_j \rightarrow \text{off-diagonal terms}$

$[J_\sigma(\vec{z})]_{ii} = \sigma(\vec{z})_i - \sigma(\vec{z})_i \sigma(\vec{z})_i \rightarrow \text{diagonal terms}$

$\Rightarrow$ In matrix form: $J_\sigma(\vec{z}) = \text{diag}(\sigma(\vec{z})) - \sigma(\vec{z})\sigma(\vec{z})^T$

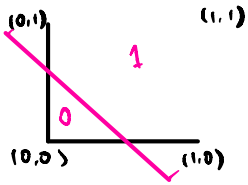
4. Simulating XOR

$$1. \quad \hat{y} = \begin{cases} 0 & \text{if } W\vec{x} + b < 0 \\ W\vec{x} + b & \text{otw.} \end{cases}$$

$W\vec{x} + b$ is a linear decision boundary on a 2-d plane with -ve values set to 0.

Thus $\hat{y}$ cannot both increase from (0,0) to (1,0) & (0,1) then decrease from (1,0) & (0,1) to (1,1).

Eg.



No matter how we draw a linear decision boundary, if (0,1) and (1,0) are both on side 1, then either (0,0) or (1,1) must also be on side 1.

Thus, $\text{ReLU}(W\vec{x} + b)$ cannot simulate XOR.

$$2. \quad h = \text{ReLU}(W_1 x + b_1) \\ \hat{y} = W_2 h + b_2$$

$$W_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad b_1 = 0$$

$$W_2 = \begin{bmatrix} 1 & 1 \end{bmatrix} \quad b_2 = 0$$

$$3. \text{ Initial state: } W_1 = 0, b_1 = 0, W_2 = 0, b_2 = 0 \\ z = W_1 x + b_1 = 0x + 0 = 0$$

$$h = \text{ReLU}(z) = 0$$

$$\hat{y} = W_2 h + b_2 = 0$$

First update:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial w_1}$$

$$\frac{\partial \hat{y}}{\partial h} = w_2^T = 0 \text{ in initial step}$$

$$\Rightarrow \frac{\partial L}{\partial w_1} = 0$$

$$\Rightarrow w_{1, \text{new}} = w_{1, \text{old}} - \eta \frac{\partial L}{\partial w_1} = w_{1, \text{old}} = 0$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial b_1} = 0$$

$$\Rightarrow b_{1, \text{new}} = b_{1, \text{old}} - 0 = 0$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} h^T = 0$$

$$\Rightarrow w_{2, \text{new}} = w_{2, \text{old}} - 0 = w_{2, \text{old}} = 0$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \cdot 1$$

$$\Rightarrow b_{2, \text{new}} = b_{2, \text{old}} - \eta \frac{\partial L}{\partial b_2} \text{ some non-zero constant}$$

Since $w_1 = w_2 = b_1 = 0$ after the initial update, we have by induction for any epoch k :

$$b_1^{(k)} = 0 \quad w_2^{(k)} = 0 \quad w_1^{(k)} = 0$$

Since w_1 and b_1 are stuck at 0, the hidden layer is also always a 0 vector. The network outputs $\hat{y} = w_2(0) + b_2 = b_2$.

The model can only learn to output constant values, but we need different outputs s.t. $(0,0) \& (1,1) \rightarrow 0$.

$(0,1) \& (1,0) \rightarrow 1$
 $\therefore$ The model cannot learn XOR.

4. (Extra credit)

Initial weights: for some $c \in \mathbb{R}$, input $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

$$w_1 = \begin{bmatrix} c & c \\ c & c \end{bmatrix} \quad b_1 = \begin{bmatrix} c \\ c \end{bmatrix} \quad w_2 = [c \ c] \quad b_2 = c$$

$$z = w_1 x + b_1 = \begin{bmatrix} cx_1 + cx_2 + c \\ cx_1 + cx_2 + c \end{bmatrix} = \mathbb{1}_{2 \times 1} \cdot c(x_1 + x_2 + c)$$

$$h = f(z) = \begin{bmatrix} f(cx_1 + cx_2 + c) \\ f(cx_1 + cx_2 + c) \end{bmatrix} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$$

* hidden neurons $h_1 = h_2$

Update step: $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial w_1}$

$$\frac{\partial \hat{y}}{\partial h} = w_2^T = \begin{bmatrix} c \\ c \end{bmatrix} \quad \frac{\partial z}{\partial w_1} = x^T$$

$$\frac{\partial h}{\partial z} = \text{diag} \begin{bmatrix} f'(cx_1 + cx_2 + c) \\ f'(cx_1 + cx_2 + c) \end{bmatrix}$$

$$\Rightarrow \frac{\partial L}{\partial w_1} = \underbrace{w_2^T \frac{\partial L}{\partial \hat{y}} \odot \begin{bmatrix} f'(z) \\ f'(z) \end{bmatrix}}_{\delta = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}} x^T$$

$$= \begin{bmatrix} \delta_1 x_1 & \delta_1 x_2 \\ \delta_2 x_1 & \delta_2 x_2 \end{bmatrix} \quad \text{and } \delta_1 = \delta_2$$

$\therefore$ Row 1 = Row 2 of $\frac{\partial L}{\partial w_1}$.

$$(*) \quad \frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} h^T \Rightarrow w_{2, \text{new}} = [c \ c] - \eta \left[\frac{\partial L}{\partial \hat{y}} h_1 \quad \frac{\partial L}{\partial \hat{y}} h_2 \right]$$

Still has identical entries as $h_1 = h_2$.

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial b_1} = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} = \begin{bmatrix} \delta_1 \\ \delta_1 \end{bmatrix} \quad \text{Still identical rows.}$$

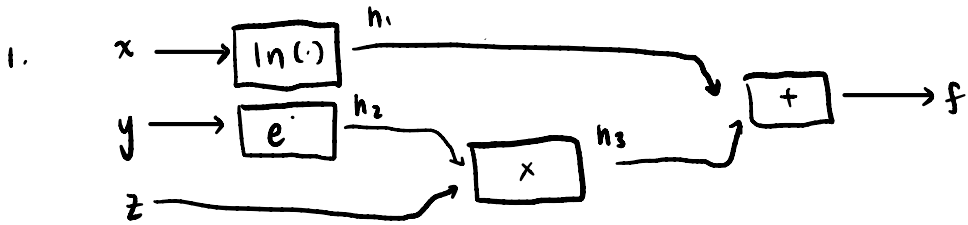
after $b_{1,\text{new}} = b_{1,\text{old}} - \eta \begin{bmatrix} \delta_1 \\ \delta_1 \end{bmatrix}$

When w_1 updates, $w_{\text{new}} = \begin{bmatrix} c+k_1 & c+k_2 \\ c+k_1 & c+k_2 \end{bmatrix}$ will have identical rows. Since the rows are identical for $w_{1,\text{new}}$ & $b_{1,\text{new}}$, then $h^{(1)}$ will have identical rows. $w_2^{(1)}$ continues to have identical rows by induction from $(*)$. Then subsequent $w_1^{(k)}$ will have identical rows too.

$$\begin{aligned} \therefore \hat{y} &= w_2 h + b_2 & w_2 &= \begin{bmatrix} w_{2,1} & w_{2,2} \end{bmatrix} \\ &= w_{2,1} h_1 + w_{2,2} h_2 + b_2 & &= \begin{bmatrix} a & a \end{bmatrix} \\ &= a h_1 + a \underbrace{h_2}_{=h_1} + b_2 \\ &= 2 a h_1 + b_2 \leftarrow \text{linear like in 4.1} \end{aligned}$$

$\therefore$ The neurons of the hidden layer are clones, so it can only solve linear problems, and thus cannot learn XOR.

5. Neural Nets and Back Propagation



2. $x=1$ $y=3$ $z=2$

$$h_1 = 0$$

$$h_2 = e^3$$

$$h_3 = 2e^3$$

$$f = 2e^3$$

3. $\frac{\partial h_1}{\partial x} = \frac{1}{x} = 1$ $\frac{\partial h_2}{\partial y} = e^y = e^3$

$$\frac{\partial h_3}{\partial h_2} = z = 2 \quad \frac{\partial h_3}{\partial z} = e^y = e^3 \quad \frac{\partial h_3}{\partial z} = h_2 = e^3$$

$$\frac{\partial f}{\partial h_1} = 1$$

$$\frac{\partial f}{\partial h_3} = 1$$

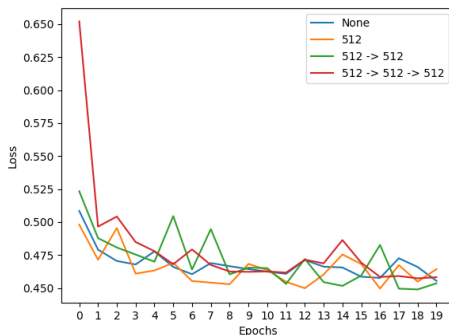
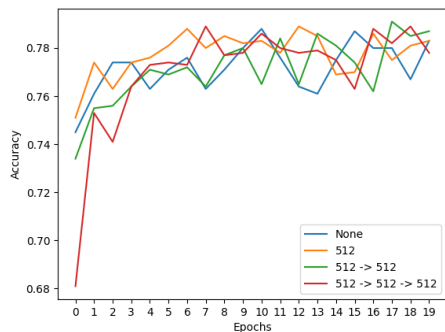
4. $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h_1} \frac{\partial h_1}{\partial x} = 1$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial y} = 2e^3$$

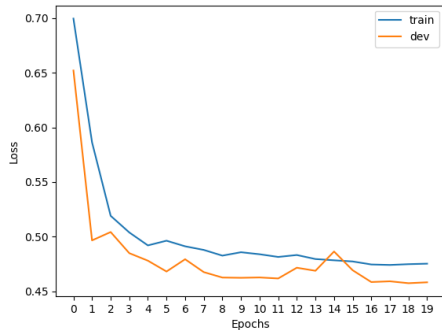
$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial h_3} \frac{\partial h_3}{\partial z} = e^3$$

6. Programming

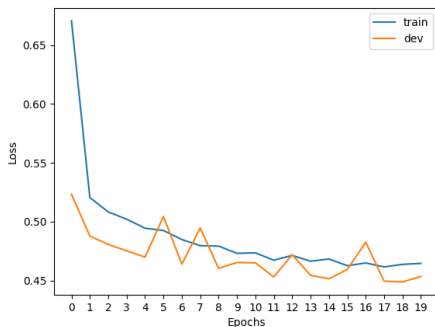
6.1.3 Train and dev loss are quite close and are consistent across different configurations. The dev loss curve is more jagged, likely due to a smaller dev set than the train set and also because the train loss is averaged over epochs while dev loss is not. Deeper models are not always better. With this example, it appears that deeper models start with lower accuracy/higher loss. This suggests that it may be harder for the gradient of the 3-layer model to find a good starting direction than a simpler model. However, all models eventually converge to ~0.8 accuracy, suggesting that the added complexity of the 3-layer model doesn't provide a significant performance boost for this specific task. In particular, if the underlying patterns in our data are relatively simple, deeper models are unnecessary and do not improve accuracy, while also making optimization harder.



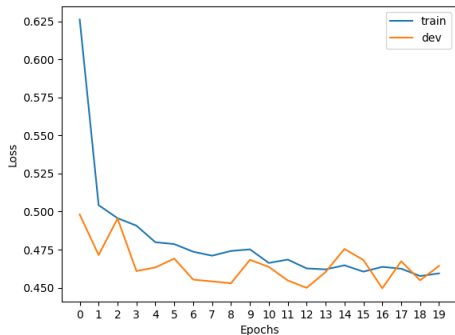
512->512->512



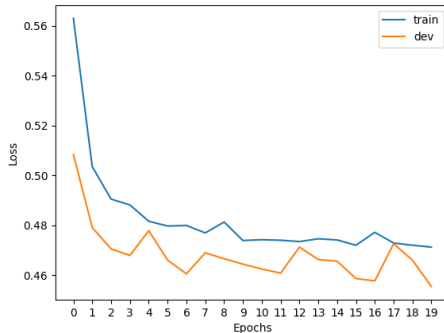
512->512



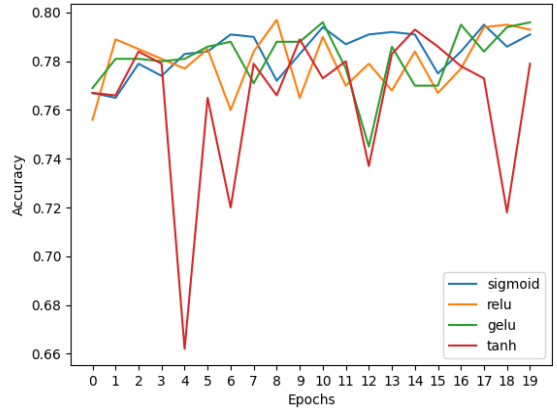
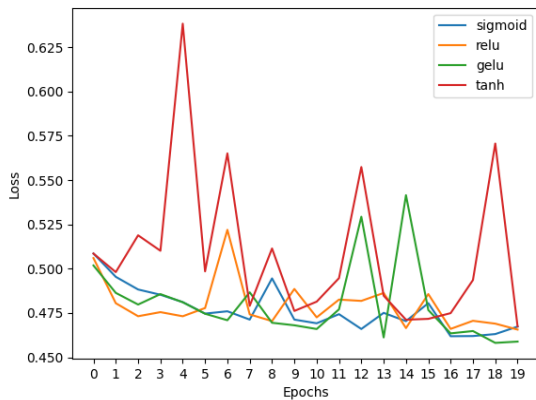
512



None



6.1.4 In our experiment to compare different activation functions in a model with 1 hidden layer, our results suggest that sigmoid, ReLU, and GeLU activation functions perform similarly, reaching an accuracy of ~ 0.79 . For this data set and task, it appears the sigmoid activation seems to have the least volatility and the tanh activation function is most volatile, with sharp drops in accuracy/rises in loss.



6.1.5 With a learning rate of 0.001, the accuracy increases/loss decreases noticeably slower than the other learning rates, but seem to almost reach the same loss and accuracy after 20 epochs. The model trained with a learning rate of 0.25 had lower losses the quickest. However, higher learning rates seem to have slightly more jagged/variable loss and accuracy curves, especially towards the later epochs, suggesting that the model might be overshooting.

